MSc Project – CSC-40098 – 2025 –Y1 –E

Alan Johnson **Student ID - 24038155**

Mentor: Beran Necat

A Comparative Evaluation of Transformer Based Language Models for the Generation of Structured Algorithmic Representations such as Pseudocode and Executable Flow Diagrams from Natural Language Algorithm Specifications.

Abstract

Transformer based language models have demonstrated substantial capability in natural language processing and increasingly in program generation and software engineering tasks, building on the attention based architecture introduced by Vaswani et al. (2017). However, the generation of syntactically plausible code or pseudocode does not, by itself, establish that the intended algorithmic semantics have been preserved. Existing code generation benchmarks such as HumanEval emphasise executable functional correctness (Chen et al., 2021), while broader benchmarking frameworks such as CodeXGLUE evaluate multiple code understanding and code generation tasks (Lu et al., 2021). More recent work, including SWE-bench, further demonstrates the importance of execution-based evaluation when assessing language models on realistic software engineering problems (Jimenez et al., 2024).

This study adopts an experimental and comparative methodology to evaluate six transformer-based language models in generating structured pseudocode and executable program representations from natural-language algorithm specifications.

To support the investigation, the Natural Language Flow Diagram Encoder Pipeline (NL-FDEP) was developed as a controlled multi stage experimental framework. Model-generated pseudocode is interpreted and canonicalised according to a defined processing profile before being transformed through to an Abstract Syntax Tree, and then to a machine-readable flow-diagram representation and finally to executable Python code. These stages follow established compiler principles in which source representations are progressively transformed into structured intermediate forms before executable output is produced (Aho et al., 2006). Intermediate artefacts are retained throughout the pipeline, allowing semantic and structural preservation to be examined across transformation stages rather than assessing only the final executable program.

Each of the six models is evaluated against the same set of 50 natural language algorithm specifications under consistent experimental conditions. A trusted 50 algorithm baseline dataset is used as a study readiness control to validate the experimental infrastructure before model generated outputs are processed. Behavioural evaluation is conducted using a frozen test framework containing Core Behavioural tests, Pre-Freeze Validation cases and Semantic Boundary Probes. Execution based testing is supplemented by automated rubric assessment and independent Human Review, providing complementary evidence rather than collapsing reliability into a single metric. Explicitly completed but non evaluable Human Reviews are incorporated through a fixed denominator failure to demonstrate rule, whereas incomplete or technically missing observations remain unresolved.

The empirical results demonstrate measurable differences in reliability across the six evaluated models at both aggregate and individual algorithm levels. Within the study dataset, Claude Sonnet achieves the highest mean behavioural score; however, per algorithm analysis shows that aggregate performance alone does not fully characterise model reliability. Variation across tasks, representation stages and assessment streams indicates that successful algorithm generation cannot be evaluated adequately through surface plausibility or a single execution score alone.

The study therefore demonstrates the value of evaluating language model generated algorithms across a complete representational pathway: natural language to pseudocode, Abstract Syntax Tree, flow diagram, executable Python code, behavioural testing and comparative analysis. The NL-FDEP provides a reproducible framework through which semantic preservation, behavioural correctness and failure points can be examined systematically across transformer based language models.

Introduction

Background

Transformer based language models have developed rapidly from systems primarily designed for natural-language processing, into general purpose models capable of performing a wide range of software development tasks. The Transformer architecture introduced by (Vaswani et al. 2017) replaced recurrent sequence processing with attention based mechanisms and provided the architectural basis for many later large language models. Increasing model scale, training data availability and code specific training have since enabled language models to perform tasks including code completion, program generation, documentation generation, code translation, debugging and natural language to code generation (Chen et al., 2021; Austin et al., 2021; Jiang et al., 2024).

The ability to generate source code from natural language descriptions changes the interface between software specification and implementation. Instead of requiring a programmer to express a solution directly in a formal programming language, a language model can interpret a natural language description of an algorithm and construct a corresponding program representation. Program generation research has long considered the problem of producing programs that satisfy some form of user supplied specification (Gulwani, Polozov and Singh, 2017). Transformer based models extend this idea by allowing comparatively unconstrained natural language to participate directly in the specification to program process.

The apparent quality of a generated artefact should not, however, be equated automatically with algorithmic correctness. Pseudocode or source code may appear syntactically convincing while containing incorrect control flow, termination conditions, boundary handling, misunderstood input requirements or other semantic defects. Conversely, a representation may differ considerably from a reference implementation while still providing the required behaviour.

Execution based benchmarks have addressed part of this problem by evaluating generated programs through behaviour rather than textual similarity alone. HumanEval evaluates generated Python code functions using functional tests (Chen et al., 2021), while Austin et al. (2021) similarly evaluate natural language to program generation using executable programming problems. These approaches demonstrate the importance of empirical execution evidence.

Final executable correctness nevertheless represents only one point in a potentially longer transformation process. Where a generated algorithm passes through several structured representations before execution, evaluating only the final program may obscure where semantic information was preserved, altered or lost. The present study therefore examines a complete transformation pathway supported by the Natural Language Flow Diagram Encoder Pipeline (NL-FDEP).

Natural Language → Pseudocode → Abstract Syntax Tree → Flow Diagram → Python code → Testing → Comparative Analysis

The intermediate representations are not treated as independent proofs of semantic equivalence. They provide observable evidence through which changes in algorithmic structure and behaviour can be traced and investigated.

Research Problem

Large language models may generate algorithmic solutions that appear plausible to a human reader but do not preserve the intended task semantics when transformed into executable form. This creates a methodological problem for comparative evaluation. Judging performance through similarity to a

reference solution may penalise legitimate alternatives, while judging performance only through final execution may conceal structural differences or transformation failures occurring earlier in the process.

The research problem is therefore the need for a controlled mechanism through which the reliability of language model generated algorithmic representations can be evaluated across multiple stages rather than at a single output point. Within the experimental scope, reliability is evidenced through successful structured processing, Core Behavioural performance, Semantic Boundary behaviour, automated rubric assessment and independent Human Review. These measures are retained separately because they capture different properties of the generated artefacts.

Research Aim

The primary aim is to comparatively evaluate the reliability of six transformer based language models in generating structured pseudocode and executable algorithmic representations from natural language algorithm specifications, using a controlled multi stage processing and evaluation framework.

The investigation therefore considers not only whether a language model can produce plausible pseudocode, but whether the algorithmic semantic meaning expressed by that pseudocode remains sufficiently preserved as the representation becomes increasingly formal and executable.

Research Objectives

- Establish a common set of 50 natural-language algorithm specifications for comparative evaluation across six transformer based language models.
- Develop a controlled multi stage pipeline capable of processing generated pseudocode through canonicalisation, parsing, Abstract Syntax Tree construction, flow diagram generation, Python code generation and behavioural testing.
- Retain intermediate artefacts so that structural and semantic changes can be investigated throughout the transformation process.
- Validate the experimental infrastructure using a trusted 50 algorithm baseline before processing experimental model outputs.
- Evaluate executable behaviour through a frozen test framework applied consistently across model datasets.
- Distinguish Core Behavioural correctness from Semantic Boundary behaviour so that out of domain responses are not conflated automatically with required task success or failure.
- Supplement execution based evaluation with automated rubric assessment and independent Human Review.
- Compare performance at aggregate and individual algorithm level and identify representative successes, failures and disagreements for closer analysis.

Research Questions

1. How reliably can the six evaluated transformer based language models generate pseudocode from natural language algorithm specifications that can then be processed into formal structured representations?

2. To what extent is the intended algorithmic structure preserved as generated pseudocode progresses through Abstract Syntax Tree, flow diagram and executable Python code representations?

3. How do six transformer based language models compare in behavioural correctness across the 50 algorithm experimental dataset?

4. What additional evidence is provided by automated rubric assessment and independent Human Review when compared with behavioural testing alone?

5. What recurring algorithm level or representation level failure patterns can be identified through analysis of the retained intermediate artefacts?

Together, these questions move the investigation beyond whether a model can simply produce source code and towards whether algorithmic meaning can remain reliable across a structured transformation pathway.

Significance and Contribution

Code generation evaluation increasingly requires methods capable of distinguishing fluent output from dependable computational behaviour. Contemporary evaluation research identifies compilation, execution, functional testing and human judgement as relevant but distinct measures of generated program quality (Chen et al., 2024; Jiang et al., 2024). Execution based benchmarks provide stronger evidence of functional correctness than textual similarity alone (Chen et al., 2021), but final execution does not necessarily reveal where a failure or semantic misalignment originated.

Different program representations expose different properties of an algorithm. Abstract Syntax Trees make syntactic structure explicit, while graph based representations can expose control relationships. Research examining source code representations; likewise identifies token based, tree based and graph based forms as complementary ways of representing programs (Samoaa et al., 2022).

The NL-FDEP applies this principle as an experimental strategy. Rather than converting model generated pseudocode directly into independently generated Python code, the algorithm is required to pass through explicit intermediate representations. This provides greater analytical resolution when a final implementation succeeds or fails.

The contribution is therefore both empirical and methodological. Empirically, six transformer-based language models are evaluated against the same 50 natural language algorithm specifications, producing 300 model algorithm observations under common downstream conditions. Methodologically, the NL-FDEP provides a reproducible framework through which generated algorithms can be transformed, executed and inspected at successive representational stages. The retained evidence makes it possible to investigate where divergence first becomes observable instead of reporting only a final pass or fail result.

Scope

This study evaluates six selected transformer based language models across 50 algorithm specifications. The task set therefore forms a defined experimental benchmark rather than the complete domain of software engineering. The principal model output under investigation is structured pseudocode generated from natural language specifications; each AI agent was given the same set of tasks worded exactly the same to prevent bias, and to ensure a consistent metric was being considered.

The final executable target is Python executable code.

Behavioural correctness is evaluated within a frozen test domain. Passing every defined test demonstrates agreement with the tested behavioural contract but does not constitute formal proof of correctness for every possible input. Human Review provides additional independent evidence but remains a human judgement process and is therefore retained separately from deterministic execution evidence.

The findings are consequently interpreted in relation to the selected models; task set, pipeline version and frozen evaluation conditions rather than as universal rankings of language model capability.

Research Methodology and Experimental Design

Experimental Design

The research adopts a controlled comparative experimental design. Empirical software engineering research requires experimental variables, treatments, measurements and procedures to be defined sufficiently clearly that observed differences can be interpreted in relation to the treatment under study (Wohlin et al., 2012).

The principal comparative factor is the transformer based language model used to generate pseudocode from a natural language algorithm specification. Six models are evaluated against the same set of 50 specifications, producing 300 model algorithm observations. Each observation consists of one model response together with the artefacts and measurements generated when that response is processed through the NL-FDEP.

Table 1 Experimental design

Element	Value
Transformer based language models	6
Natural language algorithm specifications per model	50
Primary model algorithm observations	300
Executable target	Python code
Frozen pipeline release	NL-FDEP v4.37
Active Common Tests pack	v1.4.2

The model is the principal independent variable. Dependent observations include pipeline progression, behavioural outcomes, rubric measurements, Human Review outcomes and derived comparative measures. Downstream processing is intentionally standardised so that every collected response is processed under the same pipeline and evaluation conditions.

The methodology is primarily quantitative because comparison is based on a measurable progression, behavioural and rubric outcomes. Quantitative results are supplemented by structured inspection of selected intermediate artefacts where algorithm level differences require explanation.

Experimental Inputs

These algorithms were selected from a corpus of 360 algorithms that was compiled from the 6 AI agent each agent was asked to produce a list of 60 algorithms increasing in difficulty every 12 algorithms up to level 7 difficulty. As you would expect each list contained similar responses so the decision was made, to first include only those responses that were unique while randomly choosing the same amount from each difficulty level. The final 2 algorithms were from level 7 they were chosen randomly also, this made up the 50 selected tasks giving each model the same chance of difficulty while randomising the selection throughout the models. The same 50 natural language algorithm specifications were then used across all six model datasets to minimise task selection variation. The specifications define the required behaviour while allowing each model to determine how the algorithm should be expressed. Correctness is therefore not determined through textual similarity with a trusted reference implementation; alternative algorithmic structures may be valid where they satisfy the required behavioural contract.

Once collected, model responses are treated as fixed experimental artefacts and are not manually repaired to improve downstream success. Although this study uses 50 specifications, the NL-FDEP is not intrinsically restricted to this number. Future comparative studies may employ different dataset sizes provided that the same input set and experimental conditions are applied consistently across the models being compared.

Strict Dialect Pseudocode and Canonicalisation

Pseudocode has no universally accepted syntax. Equivalent operations can be expressed using different vocabulary, punctuation, indentation conventions and natural language phrases. This flexibility is useful for human communication but creates ambiguity where pseudocode becomes direct input to a deterministic parser.

The NL-FDEP therefore uses a strict pseudocode dialect adapted from the pseudocode conventions defined in the *Cambridge International AS & A Level Computer Science 9618 Pseudocode Guide for Teachers* (Cambridge Assessment International Education, 2023). The dialect is used to minimise unnecessary representational variation between model responses and to provide a predictable interface for machine processing. The purpose is not to force every model to generate the same algorithm. It is to constrain the representation of common operations while leaving the model responsible for the algorithmic reasoning itself.

Before parsing, generated pseudocode is interpreted and canonicalised according to a defined processing profile. Canonicalisation may normalise recognised representational differences, but it must not invent missing operations, conditions or control structures, nor transform an incorrect algorithm into a correct one.

Representation may be normalised, but algorithmic meaning must not be invented.

The original and processed pseudocodes are retained separately, together with the processing report, so that every compatibility transformation remains auditable. This distinction protects the experiment from becoming a measure of how effectively the infrastructure can repair a models response.

Multi-Stage Processing

Pseudocode → AST → Flow Diagram → Python code → Behavioural Testing

Following canonicalisation, the parser transforms accepted pseudocode into an Abstract Syntax Tree. The AST is then encoded into a machine readable flow diagram representation, and that becomes the input to deterministic Python code generation. The generated Python code is subsequently executed against the frozen behavioural tests.

Python code is deliberately generated from the flow diagram representation rather than directly from the original pseudocode. This makes the intermediate representations part of the executable dependency

chain. A semantic or structural defect introduced or exposed earlier can therefore propagate into the final executable representation and become visible during testing.

Trusted Baseline and Study-Readiness Gate

Before experimental model outputs are evaluated, a trusted dataset containing the corresponding 50 algorithms is processed through the frozen infrastructure. The baseline acts as a study readiness control. Its purpose is to establish that the NL-FDEP can process the defined algorithm set under trusted conditions before model-generated failures are interpreted experimentally.

The baseline is not used as a runtime correctness oracle. An experimental response is not declared correct because it resembles the baseline and is not declared incorrect because it differs from it. This preserves the possibility of alternative valid algorithms.

The baseline therefore answers whether the experimental infrastructure can process a trusted implementation of the study problem; the frozen tests and evaluation procedures determine whether the model generated implementation demonstrates the required behaviour.

Frozen Experimental Conditions

Comparative evaluation requires stable processing conditions. Changing parser behaviour, compilation rules, test cases or scoring procedures between models datasets could introduce systematic differences unrelated to the models datasets themselves. The relevant processing and evaluation components are therefore frozen for the experimental run.

The study reported here uses the NL-FDEP v4.37. Later software revisions may improve the implementation, but results generated under different pipeline behaviour are not mixed within the same comparative dataset unless the complete dataset is reprocessed under the new conditions.

Behavioural Evaluation

Behavioural correctness is evaluated by executing generated Python code against the frozen Common Tests v1.4.2 framework. The pack contains 50 algorithm suites and 156 declared test cases. The declared composition is shown in Table 2. The pack metadata records 99 enabled and 30 disabled Core Behavioural cases, all 8 Pre-Freeze Validation cases enabled, and 17 of the 19 Semantic Boundary Probes enabled under the frozen configuration.

Table 2 Common Tests v1.4.2 composition

Category	Declared	Enabled	Role in study
Core Behavioural	129	99	Principal behavioural scoring surface
Pre-Freeze Validation	8	8	Validation evidence; excluded from Core Behavioural score
Semantic Boundary Probes	19	17	Boundary behaviour; excluded from Core Behavioural score
Total	156	124	Frozen Common Tests pack

Core Behavioural tests assess whether the implementation performs the required algorithmic task within the defined specification. Pre-Freeze Validation cases are retained as validation evidence and do not alter the frozen core scoring surface. Semantic Boundary Probes examine behaviour outside or at the edge of the defined semantic contract.

An implementation accepting an out of domain input is not necessarily more correct than one rejecting it where no behaviour is specified. Boundary outcomes are therefore reported separately as Semantic Boundary Variance rather than being incorporated automatically into the Core Behavioural score.

A passing behavioural outcome provides direct evidence that the generated implementation produced the expected result for that frozen test case. Passing every supplied test does not constitute formal proof of universal semantic equivalence; the strength of the approach lies in providing a consistent and reproducible operational measure across all supplied model datasets. This interpretation is consistent with execution based evaluation used in benchmarks such as HumanEval and SWE-bench (Chen et al., 2021; Jimenez et al., 2024).

Automated Rubric Assessment and Independent Human Review

Behavioural correctness does not capture every property relevant to the structured transformation process. Automated rubric assessment is therefore retained as a separate evidence stream. It evaluates predefined properties of the generated artefacts and produces structured measurements that can be compared across model algorithm observations.

Independent Human Review provides a complementary assessment of the generated artefacts and rubric criteria. It is not treated as an informal mechanism for overwriting the automated score. Automation provides consistency and scalability, while Human Review can identify properties that are difficult to encode fully in deterministic rules. Agreement between the two streams provides converging evidence; disagreement becomes an analytical result that can be investigated through the retained artefacts.

Human Review Terminal Outcomes

A completed Human Review does not always produce a legitimate numerical rubric percentage. Treating every absent score as zero would conflate completed non evaluable cases with genuinely incomplete or technically missing evidence. Conversely, excluding a completed non-evaluable case from the denominator would allow a terminal failure to demonstrate evaluable performance to disappear from the study-level mean.

For algorithm i, the Human Review contribution is defined as follows:

$$H_i = \begin{cases} h_i, & \text{completed and numerically scorable,} \\ 0, & \text{completed and explicitly non-evaluable,} \\ \text{unresolved}, & \text{incomplete or technical/data missingness.} \end{cases}$$

The zero credit condition applies only when the review is explicitly adjudicated as HUMAN_REVIEW_COMPLETE_NOT_EVALUABLE. A missing percentage without that terminal status remains unresolved.

Fixed-Denominator Failure-to-Demonstrate Rule

When all N algorithms have terminal Human Review outcomes, the Human Review Overall is calculated using the fixed denominator:

$$H_{\text{overall}} = \frac{1}{N} \sum_{i=1}^{N} H_i, \qquad \text{when all } N \text{ algorithms have terminal outcomes.}$$

This is a fixed denominator failure to demonstrate rule rather than missing value imputation. A completed non evaluable review is a recorded terminal outcome; an incomplete or technically corrupted review is insufficient evidence and cannot legitimately be converted to zero. The overall calculation is therefore withheld while unresolved cases remain.

For transparency, the numerical only Human Review mean is also retained. For example, two numerical scores of 90 per cent and one completed non evaluable outcome produce:

$$\frac{90 + 90 + 0}{3} = 60\%, \qquad \frac{90 + 90}{2} = 90\%.$$

The fixed denominator Human Review Overall is therefore 60 per cent, while the numerical only, audit mean remains 90 per cent. Reporting both values makes the effect of the completed non evaluable outcome visible.

Evidence Collection, Comparative Analysis and Reproducibility

The study deliberately avoids collapsing reliability into a single opaque measure. The analytical framework retains pipeline and representation outcomes, Core Behavioural performance, Semantic Boundary Variance, automated rubric assessment and independent Human Review as distinct evidence streams.

For each model algorithm observation, the NL-FDEP retains the relevant source and generated artefacts, including model generated pseudocode, processed pseudocode, processing and parser reports, AST, machine readable flow diagram representation, visual flow diagram artefacts, generated Python code, compilation information, behavioural outcomes and study scoring information where applicable. Retention provides both an audit trail and a route from aggregate results back to individual algorithms.

The principal analysis is descriptive and comparative. Aggregate model level measures identify overall patterns, while algorithm level observations reveal whether those patterns are consistent across tasks or driven by particular cases. The 50 algorithms form a defined benchmark rather than a random sample from a formally specified population of all possible algorithms; broader generalisations have not been considered.

Reproducibility is supported by pipeline versioning, frozen Common Tests, retained artefacts, machine readable intermediate representations, explicit scoring rules, recorded Human Review terminal states and deterministic downstream transformations where possible. Reproducibility therefore means not only recalculating an aggregate score, but also reconstructing the sequence of evidence from which that score was derived, traceability is a key design consideration.

Validity and Responsible Research

Internal validity is supported by applying the same 50 specifications, frozen downstream pipeline and corresponding Common Tests across all six model datasets, together with a trusted baseline used to establish study readiness. Pipeline transformations and canonicalisation are recorded; and automated and Human Review evidence remains distinct.

Construct validity is addressed by operationalising reliability through multiple measures rather than equating it with a single score. Core Behavioural performance captures tested functional behaviour; intermediate representations provide structural evidence; automated assessment provides predefined

artefact level measures; Human Review supplies independent evaluative evidence; and Semantic Boundary Probes examine behaviour outside the principal contract.

External validity remains bounded by the study configuration. The findings concern six models, 50 algorithm specifications and a Python code executable target. Interactive debugging, repository scale development and large multi file systems introduce different challenges and fall outside the experimental scope.

Ethical and Responsible Research Considerations

The principal research objects are generated software artefacts rather than personal or sensitive human data. Human Review is limited to generated artefacts and predefined assessment criteria, and the study does not profile individual users. The findings therefore cannot be associated with any single individual.

Responsible reporting nevertheless requires transparency regarding the limitations of language model evaluation. Hosted language models can change over time, and performance may vary according to model version, task selection and experimental conditions. The comparative findings are therefore reported in relation to the collected datasets and frozen study configuration rather than as permanent or universal rankings of the evaluated models. Original model outputs and downstream artefacts are preserved where permitted to support research transparency, reproducibility and academic integrity.

The NL-FDEP Architecture and Design Rationale

Architectural Overview

The NL-FDEP was developed to implement the experimental methodology rather than as an independent software development exercise. The language model is responsible for interpreting the natural language requirement and generating pseudocode; the NL-FDEP begins processing from that pseudocode and determines whether the representation can be transformed through increasingly formal stages while retaining sufficient structure and behaviour.

Pseudocode → Processed Pseudocode → AST → Flow diagram representation → Python code → Test Results

The pipeline is divided into explicit stages rather than operating as a single end to end transformation. This provides two principal benefits. First, a final result can be traced backwards through the representations that produced it. Second, a failure can be localised more precisely than if the system returned only a final program.

The architecture therefore prioritises consistent processing, deterministic transformations where possible, preservation of the model generated algorithm, traceability between stages and the ability to identify where a divergence first becomes visible.

Natural Language Specification and Model Output

The natural language algorithm specification is the common experimental input supplied to each model. It defines the required task without prescribing one exact pseudocode implementation. This permits alternative valid algorithms while maintaining a consistent problem statement across the six datasets.

The NL-FDEP does not use the trusted baseline as a template for the model output. A response can differ substantially in structure or vocabulary from a trusted implementation and still be behaviourally valid. The model generated pseudocode is therefore treated as the primary artefact entering the pipeline.

Strict Dialect Pseudocode

The NL-FDEP uses a strict pseudocode dialect to reduce variation in surface representation and to provide a sufficiently predictable input for deterministic parsing. Without such a boundary, the parser would need either to accept an increasingly broad set of informal forms or to infer the meaning of expressions it had not been designed to process. Both alternatives would weaken experimental control. A highly permissive parser could interpret superficially similar responses differently, while an inference heavy parser could begin altering the algorithm under evaluation. The strict dialect therefore restricts notation while retaining algorithmic freedom. Common operations such as assignment, input, output, conditions and loops are represented using forms that can be interpreted consistently. The study is interested primarily in the algorithms generated by the models, not in whether equivalent output behaviour is described using PRINT, DISPLAY or an unrestricted natural language sentence. The strict dialect is the concrete implementation used by the present study. Its broader conceptual implications, including whether deliberately restricted pseudocode like forms might motivate a Controlled Intermediate Language, are considered in the Discussion Section rather than assumed as part of the original methodology.

Pseudocode Processing and Defined Profiles

Language model output cannot be assumed to conform perfectly to the strict dialect in every instance. The NL-FDEP therefore contains a processing stage before parsing. A defined processing profile specifies the forms that are recognised and the transformations that may legitimately be applied. The processor can resolve supported representational variation where the intended construct is already present. It cannot add missing algorithmic operations, conditions or control structures. Allowing such repair would cause the final behavioural score to reflect the corrective ability of the infrastructure rather than the original model

output semantic objectivity. The original pseudocode, processed pseudocode and processing report are retained separately. This makes canonicalisation auditable and also allows the degree of representational adjustment required by different model outputs to be examined when relevant.

Deterministic Parsing and Abstract Syntax Tree

After pseudocode processing, the parser converts the accepted text into a structured machine readable representation. The parser is intentionally deterministic because it forms part of the experimental infrastructure rather than the comparative variable. For the same accepted pseudocode and pipeline configuration, the structural interpretation should remain stable.

The parser produces an Abstract Syntax Tree. ASTs are established compiler representations through which program structure can be made explicit independently of surface formatting (Aho et al., 2006). Within the NL-FDEP, the AST confirms that the processed pseudocode has been interpreted formally, exposes constructs such as assignments, decisions, loops, inputs and outputs, and provides the source representation for subsequent flow diagram generation.

The machine readable AST is retained rather than discarded after flow generation. If a later flow diagram representation appears incorrect, the AST provides an intermediate point at which the preceding parsing stage and the subsequent graph transformation can be compared. A visual AST is additionally produced for inspection, but the machine readable representation remains the computational artefact.

AST to Machine Readable Flow diagram representation

The AST is transformed into a machine readable graph and adjacency list representing control flow. Whereas the AST primarily reflects hierarchical program structure, the flow diagram representation makes execution relationships explicit. Sequences, decisions, branches, loops and termination paths are expressed through connected nodes.

Control flow is central to algorithmic meaning. An algorithm containing the correct individual operations in the wrong execution order is not equivalent to the intended algorithm and an incorrect branch or loop destination can change behaviour even where node contents appear valid. The flow diagram representation therefore provides another level at which structural preservation can be examined.

A distinction is maintained between the machine readable graph and its visual presentation. Execution relationships are defined by the graph itself; screen position, colour and connector routing are presentation choices and do not determine program semantics.

Visual Flow and Draw.io Representations

The NL-FDEP produces human readable visualisations of the underlying graph, including HTML and Draw.io compatible representations. These visual forms support inspection and explanation while remaining derived from the authoritative machine readable structure.

The Draw.io representation also provides a portable and editable format outside the core interface. The same underlying graph can therefore be represented as machine readable data, an HTML visualisation or a Draw.io diagram without changing its intended computational relationships.

Flow Diagram as an Alternative Input

Although the comparative experiment begins with model generated pseudocode, the NL-FDEP also supports a structured flow diagram as an alternative input route. A suitable Draw.io representation can be imported, converted into a graph, validated and used for Python code generation and behavioural testing.

The Tic-Tac-Toe example demonstrates this pathway. Its significance is architectural rather than comparative: it shows that the system can enter the transformation process at a later representational stage and is therefore concerned more broadly with transformations between structured representations of algorithmic meaning.

This capability also opens a wider question for subsequent discussion. If both strict pseudocode and a controlled graphical representation can provide machine processable descriptions of an algorithm, the important issue may be the properties required of the interface between human algorithmic intent and formally defined machine structure, rather than whether that interface is necessarily textual.

Flow Validation and Deterministic Python code generation

Before code generation, the flow graph is validated to ensure that it contains a usable set of nodes and relationships. Stage specific validation prevents a malformed graph from proceeding silently into compilation and later being interpreted as though the Python code generator introduced the problem.

Python code is generated from the validated flow diagram representation rather than directly from the original pseudocode. This is a central design decision. If the flow diagram were generated only for display purposes and Python code were compiled independently from pseudocode, successful execution would provide little evidence that the intermediate flow diagram representation had preserved the algorithm.

Pseudocode → AST → Flow diagram representation → Python code

The dependent chain makes earlier structural loss potentially observable downstream. For example, an incorrect loop relationship in the flow graph may affect generated Python code and become visible

through behavioural testing. Python code generation is deterministic and does not invoke another language model. Supported flow constructs are translated using defined compiler behaviour, avoiding the introduction of a further probabilistic transformation. Under the same NL-FDEP version, an identical validated flow diagram representation should therefore produce the same generated Python code.

Executable Target, Compilation and Behavioural Testing

Python code was selected as the executable target because its syntax is comparatively clear, it supports the constructs required by the task set and it can be executed conveniently within the behavioural testing framework. The architecture is not conceptually restricted to Python code, but the present study evaluates only this target. Generated Python code is syntax and compilation checked before behavioural execution. Compilation failure and behavioural failure are retained as different outcomes. A program that cannot execute has failed to reach a valid executable representation; a program that executes but produces incorrect results has demonstrated a different type of failure. Once executable, the program is evaluated against the corresponding frozen Common Tests. Individual test outcomes are retained rather than reducing the evidence immediately to a single percentage, allowing later analysis to identify which behaviours succeeded or failed.

Artefact Retention, Processing Reports and Failure Localisation

The NL-FDEP retains the original and processed pseudocode, processing report, parser report, AST, AST visualisation, machine readable and visual flow diagram representations, Draw.io representation where applicable, generated Python code, compilation information and behavioural test evidence. These artefacts form an evidence chain rather than merely a collection of implementation outputs.

Processing reports record the decisions made at individual stages and help distinguish a model generated problem from an infrastructure or data issue. Explicit stage boundaries also allow failure localisation. A response rejected before AST construction is analytically different from a response that produces valid formal representations but later fails behavioural testing. The retained evidence therefore links quantitative comparison with qualitative investigation: an unusual aggregate result can be traced to an individual algorithm and then backwards through the representations that produced it.

Separation of Experimental Evidence and Visualisation

The NL-FDEP separates evidence generation from evidence presentation. Raw artefacts, reports and scores form the experimental evidence; dashboards aggregate and visualise that evidence but do not execute the Common Tests, alter scores or retrospectively determine correctness. The comparative heat map supports identification of model level and algorithm level patterns. The comparative radial dashboard provides an

aggregate view and determines its centre benchmark dynamically from the model with the highest mean behavioural score in the loaded data. Algorithm level radial profiles provide a route from aggregate performance back to the 50 individual tasks. The final scoring dashboard brings together behavioural, automated rubric and Human Review evidence while preserving their separate meanings.

Version Control, Extensibility and Representational Implications

The NL-FDEP is versioned so that the experimental infrastructure associated with a study can be identified. The present results were produced with v4.37; later versions may correct defects or extend functionality but do not retrospectively replace the frozen experimental configuration.

The architecture is not intrinsically limited to six models, 50 specifications or a single executable target. Different comparative datasets or target compilers could be introduced, but a change to a major processing stage would create a new experimental configuration requiring independent validation.

The pipeline also places its representations at different points between human readability and machine formality. Natural language is highly flexible, strict pseudocode is human-readable but constrained, the AST is primarily machine oriented, the flow diagram representation has both computational and visual forms, and Python code is simultaneously readable and executable. The experience of using strict pseudocode and a structured flow-input route provides the basis for later consideration of a more general controlled intermediate representation without redefining the current strict dialect as such.

Pipeline Validation and Study Readiness

Validation as a Prerequisite

Before the experimental language model datasets could be interpreted, the NL-FDEP itself had to demonstrate sufficient reliability to act as the experimental instrument. A defect in the parser, flow encoder, Python code generator, behavioural test harness or scoring system could otherwise be mistaken for a model failure.

Validation was therefore separated into two related questions: whether the pipeline operated according to its implementation and design requirements, and whether the frozen configuration could process the specific algorithm set used by the study. Both technical integrity and study readiness were required before comparative evaluation.

Separation of Development and Experimental Evaluation

The NL-FDEP was developed iteratively, but continued modification during comparative processing would have introduced an uncontrolled variable. Development and validation were therefore completed before the study configuration was frozen. All six model datasets were subsequently processed using the same pseudocode rules, parser behaviour, flow transformations; Python code generation, tests and scoring methodology.

The frozen study configuration is NL-FDEP v4.37. Later releases may improve the software, but those changes are treated as subsequent versions rather than introduced selectively into the existing dataset.

Automated Regression Validation

A substantial automated regression suite was maintained during development to detect unintended changes across interacting components. This is particularly important in a multi-stage system because a change to pseudocode processing may affect parsing, an AST change may alter graph structure, and a flow diagram change may affect generated Python code and behavioural outcomes. Systematic software testing therefore forms part of the reliability evidence for the experimental instrument (Ammann and Offutt, 2016). The final v4.37 validation results are summarised in Table 3.

Table 3 THE NL-FDEP v4.37 automated validation

Validation surface	Result
Pytest	1,377 passed; 3 skipped; 5 subtests passed; 4 warnings
Unittest	481 passed; 1 skipped
Verified checksum files	1,087
Known pre-existing checksum mismatch	flow diagram components/flow_diagram.py
Active Common Tests pack	v1.4.2: 50 suites; 156 declared cases

These results provide evidence that the behaviours exercised by the regression suites operated successfully in the frozen release. They do not prove that the software is incapable of containing defects outside the tested domain; automated testing provides bounded evidence rather than universal proof.

Stage-Specific and Transformation Validation

Validation is performed at individual processing stages as well as end-to-end. Pseudocode processing records canonicalisation status, the parser records whether the representation is accepted structurally, AST construction can be inspected independently of graph generation, the flow graph is validated before compilation, and generated Python code is syntax checked before behavioural execution.

This staged approach is necessary because an end to end failure alone would provide insufficient information about where a technical problem originated. Transformation boundaries are equally important: successful parsing does not establish automatically that every AST relationship has been encoded correctly in the flow graph, and a valid graph does not independently prove that every branch or loop has been preserved during Python code generation. The retained intermediate artefacts therefore contribute both to empirical analysis and to validation of the experimental infrastructure itself.

End-to-End Validation

Two worked examples provide compact end to end demonstrations of the supported input routes. The principal pseudocode path example represents a summation algorithm and progresses through pseudocode processing, parsing, AST generation, flow generation, Python code generation and behavioural execution. Four demonstration cases are supplied and all four pass.

The alternative Draw.io route is demonstrated using Tic-Tac-Toe. The imported diagram, is validated as a graph, converted into executable Python code and exercised using two demonstration scenarios: a Player 1 win and a complete nine-move draw. Both behavioural demonstrations pass.

Table 4 End-to-end demonstration evidence

Route	Structural evidence	Behavioural evidence	Status
Pseudocode input	Complete pseudocode → AST → flow → Python code chain	4/4 demonstration tests passed	PASS
Draw.io flow input	35 shapes; 50 connectors imported and validated	2/2 behavioural runs passed	PASS

These demonstrations are separate from the frozen Common Tests used for comparative scoring. Their purpose is to show that the supported transformation routes can operate end to end, not to contribute to model performance results.

Package and Artefact Integrity

The frozen release archive was checked for structural integrity, and checksum verification was used to identify unexpected file changes. The v4.37 validation record reports 1,087 verified checksum files. One known pre-existing mismatch affecting flow diagram components/flow_diagram.py was recorded explicitly rather than hidden. Package integrity and functional correctness are distinct concerns. A checksum cannot demonstrate that a parser behaves correctly, but it can identify whether a file differs from the expected release state. Functional behaviour of the known mismatch remained covered by the regression suite.

Trusted Baseline and Study Readiness

Regression testing establishes confidence in the implementation but does not establish that the pipeline can process every algorithm selected for the comparative experiment. The trusted 50 algorithm baseline addresses this study specific question. The most recent fresh frozen-baseline calibration preceding v4.37 recorded pipeline SUCCESS for all 50 algorithms, all 50 calibrations completed, and a mean baseline calibration of 100 per cent. The v4.37 current check subsequently accepted all 50 prior baseline algorithms as current under matching semantic fingerprints, with zero reprocessed. The active release therefore retained the validated 50 algorithm readiness state without introducing a new semantic baseline.

Table 5 Study-readiness baseline evidence

Measure	Result
Baseline algorithms	50
Pipeline status	50/50 SUCCESS
Calibration status	50/50 CALIBRATED
Mean baseline calibration	100.0%
v4.37 currentness check	50/50 accepted as current; 0 reprocessed

The readiness condition is procedural: experimental model evaluation should not begin until the infrastructure has demonstrated that it can process the defined workload under trusted conditions. If a trusted baseline case fails, the infrastructure must be investigated before an equivalent experimental failure can be attributed to a model.

Validation of the Common Tests and Scoring

The Common Tests form part of the measurement system and were therefore validated before freezing. An incorrect expected result could fail a valid implementation, while a weak test could allow incorrect behaviour to appear successful. The separation between Core Behavioural, Pre-Freeze Validation and Semantic Boundary cases was established before comparative analysis so that test categories could not be altered retrospectively in response to the observed models results. Scoring was validated independently of the visual dashboards by supplying known inputs and comparing the calculated outputs with the expected results. Particular attention was given to the fixed denominator Human Review rule. The implementation was also tested to confirm that an absent Human Review score without explicit terminal adjudication remains unresolved rather than being converted automatically to zero.

Dashboard Validation and Separation of Evidence

The analytical dashboards were checked against the underlying study evidence. Their role is to display and aggregate measurements rather than generate experimental outcomes. The comparative heat map is derived from model level and algorithm-level measurements, while the radial dashboard determines its

global centre from the highest observed mean behavioural score in the loaded data. An exact tie is reported as joint highest rather than forcing a single winner. The experiment therefore remains interpretable independently of the dashboards. Behavioural results, rubric values and Human Review outcomes exist as retained evidence before visualisation and can be verified without relying on a presentation layer.

Human Review as an Independent Validation Stream

The regression suite and trusted baseline reduce technical risk but do not make human inspection unnecessary. Human review serves as a check and balance metric, during Human Review of the v4.37 study artefacts, two issues became visible because automated assessment disagreed with inspection of the generated representations. The first concerned reachability in some nested structures. Nested loops and methods contained within class structures could be reported as unreachable in the JSON assessment even where further inspection showed that the flow diagram representation itself was correct. The second concerned AST to flow node count comparison. Some AST nodes are intentionally removed because they do not serve a corresponding purpose in the flow diagram, yet the differing node counts in some cases were reported automatically as failures. Where inspection established that the generated representation was correct, Human Review recorded disagreement with the automated assessment rather than treating the automated result as authoritative. These issues were subsequently addressed in v4.38. The empirical results reported here remain those produced under v4.37; the later fixes are not applied retrospectively. These cases demonstrate why Human Review is valuable as an independent evidence stream. Disagreement can expose limitations in the automated measurement process without converting Human Review into an informal mechanism for selectively rewriting experimental results.

NOTE: this study was undertaken with v4.37 and it is those results that are being reported. In the cases were the pipeline was incorrectly reporting an error due to an internal issue the accepted results are below that of what they should be in the final calculation as an average will be being reported and are addressed in v4.38.

Remaining Validation Limits and Transition to Experimentation

Validation does not imply that the NL-FDEP can process every possible pseudocode program or algorithm. The frozen system is intentionally bounded by its supported dialect, grammar, graph structures and executable target. A response may be understandable to a human reader while falling outside the controlled representation accepted by the parser. Such a rejection does not demonstrate that the model is universally incapable of solving the underlying problem. It demonstrates that the returned representation could not be processed successfully under the interface defined by this experiment. Once the regression

suite, trusted baseline, behavioural calibration and scoring checks satisfied their requirements, the configuration was treated as ready for comparative processing. From this point onwards, model specific failures were not used as opportunities to modify the pipeline selectively. Failures became part of the empirical dataset under a common frozen instrument.

Experimental Results

Experimental Dataset

The comparative experiment consists of six transformer based language model datasets, each containing responses to the same 50 natural language algorithm specifications. The primary dataset therefore contains 300 model algorithm observations. Each observation begins with the pseudocode generated by one model for one specification and, where processing succeeds, includes the downstream structured and executable artefacts generated by the NL-FDEP.

The results are considered at both model and algorithm level. Pipeline progression identifies whether a response can enter and traverse the formal transformation stages, while Core Behavioural testing provides the principal execution based comparison. Semantic Boundary Variance, automated rubric assessment and Human Review remain separate evidence streams.

Representation Level Outcomes

Structural process ability and behavioural correctness are related but not interchangeable. A model response can be canonicalised, parsed and converted through AST, flow diagram representation and Python code generation while still failing to demonstrate the required behaviour. Conversely, a response that cannot be converted into the required formal structure cannot produce behavioural evidence through the principal execution pathway. For this reason, stage outcomes are interpreted according to the point at which successful transformation can no longer be demonstrated. A parser rejection, compilation failure and behavioural test failure therefore represent different empirical outcomes rather than one undifferentiated error category.

Core Behavioural Performance

Core Behavioural testing provides the principal functional comparison across the six model datasets. The frozen test framework applies the same task specific cases to corresponding model outputs, allowing differences in observed executable behaviour to be compared under common conditions.

The study data demonstrates measurable differences between the six evaluated models. At aggregate level, Claude Sonnet records the highest mean behavioural score across the 50-algorithm benchmark. This

identifies Claude Sonnet as the strongest aggregate behavioural performer within the collected dataset and frozen study configuration. The ranking is not interpreted as a permanent or universal property of the models. Hosted systems can change over time, and the observed result depends on the selected tasks, model versions, prompting conditions and experimental infrastructure. The supported conclusion is therefore comparative: Claude Sonnet achieves the highest mean behavioural performance in the current study data.

Algorithm-Level Variation

The aggregate model means do not describe the complete performance distribution. Results vary across the 50 individual algorithms and the strongest model overall is not necessarily the strongest model on every task. Some algorithms produce similar outcomes across all six models, whereas others generate a wider spread of behavioural results. This variation is analytically important because an aggregate mean can conceal isolated weaknesses or task specific strengths. Algorithms producing a large cross model spread provide greater discrimination between model behaviours, while algorithms on which all models perform poorly may indicate common areas of difficulty. These cases form the basis for artefact level investigation in the Discussion rather than being explained solely from the aggregate ranking.

Semantic Boundary Variance

Semantic Boundary Probe outcomes remain separate from the Core Behavioural score. The probes examine inputs or conditions at or beyond the explicit task contract, where different implementations may make different assumptions without one response necessarily being more correct than another. A high Core Behavioural result therefore does not imply that a model will behave identically to other models at the semantic boundary. Equally, consistent boundary handling does not compensate for failure to meet the required Core Behavioural contract. Boundary behaviour is treated as an additional descriptive result rather than being used to alter the principal behavioural ranking.

Automated Rubric and Human Review Evidence

Automated rubric assessment and Human Review provide additional perspectives on the generated artefacts. Their purpose is not to replace behavioural testing or to be averaged invisibly into one model quality figure. Instead, the evidence streams allow agreement and disagreement to be identified.

Where behavioural, automated and Human Review evidence converge, confidence in the observed interpretation is strengthened. Where they diverge, the disagreement identifies an algorithm or representation that requires closer examination. The v4.37 Human Review findings concerning flow

reachability and AST to flow node counts demonstrate why such disagreement can itself be methodologically useful.

Comparative Visualisation

The comparative heat map and radial dashboards provide different views of the same retained study evidence. The heat map exposes the distribution of results across the 50 algorithms and supports identification of common strengths, weaknesses and outliers. The model level radial dashboard provides an aggregate comparison, while the 50 algorithm radial profiles show how individual task results contribute to the broader model outcome. Within the current study data, the radial dashboard identifies Claude Sonnet as the model with the highest mean behavioural score. Algorithm level profiles nevertheless show that this aggregate position is not reproduced uniformly across every task.

Model reliability varies by algorithm; aggregate performance alone does not fully characterise the observed behaviour.

Relationship to the Research Questions

RQ1 is addressed through pipeline progression and stage specific outcomes, which show how reliably model generated pseudocode can enter and traverse the formal transformation process.

RQ2 is supported by the retained AST, flow diagram representation and generated Python code artefacts, which allow structural preservation to be inspected and provide the evidence needed to determine where divergence first becomes observable.

RQ3 is addressed primarily through Core Behavioural performance. The study data demonstrates differences across the six models, with Claude Sonnet recording the highest mean behavioural score within the current benchmark.

RQ4 is addressed through the additional evidence supplied by automated rubric assessment and independent Human Review. Agreement and disagreement between these streams provide information that is not available from behavioural execution alone.

RQ5 is addressed through algorithm level comparative views and retained artefacts, which identify recurring and exceptional cases suitable for detailed failure pattern analysis.

Selection of Cases for Discussion

The quantitative and stage level findings provide a basis for selecting representative cases for deeper analysis. Selection is driven by the observed evidence rather than by whether an example favours a particular model. Useful cases include a complete end to end success; successful structural processing followed by behavioural failure; notable canonicalisation, disagreement between automated and Human

Review, substantial cross model variation on the same algorithm, and an outlier where the aggregate model ranking does not predict the individual result.

Each selected case can be followed through the retained evidence chain from natural language specification to pseudocode, processed pseudocode, AST, flow diagram representation, Python code, behavioural outcome and assessment evidence. This creates the basis for examining where semantic alignment is preserved, where divergence becomes visible, and what those observations imply for the use of structured intermediate representations in language model assisted program generation.

Experimental Results

Dataset Completeness and Study Coverage

The completed comparative dataset contains all 300 planned model algorithm observations: six transformer based language models evaluated against the same 50 algorithm specifications. The dataset consistency check reported no missing or additional algorithm identifiers across the six datasets, so the comparison is based on a complete and aligned

6 × 50 design.

Across the 300 observations, 225 produced evaluable semantic behavioural scores and 75 were recorded as not evaluable. Of the 225 evaluable observations, 179 were at the frozen semantic reference and 46 were below it. This means that 75.0% of all planned observations were behaviourally evaluable and, within that evaluable subset, 79.6% reached the reference level. The remaining 25.0% of observations are important because they are not equivalent to low behavioural scores: they represent cases for which the required behavioural comparison could not be established under the frozen v4.37 experimental pathway.

Table 6 Study coverage and evaluability

Measure	Count	Proportion
Planned model algorithm observations	300	100.0%
Behaviourally evaluable observations	225	75.0%
Not evaluable observations	75	25.0%
At frozen semantic reference	179	59.7% of all observations / 79.6% of evaluable
Below frozen semantic reference	46	15.3% of all observations / 20.4% of evaluable

Source: NL-FDEP Interactive Comparative Heat Map, completed study dataset.

Comparative Heat Map Results

The comparative heat map provides the first complete model level comparison. It reports two automated measures that answer related but different questions. Mean semantic behavioural score is the mean of the algorithm level semantic behavioural scores for evaluable algorithms. Aggregate Common Test accuracy

pools the individual Common Test outcomes. Because some algorithm suites contain more test cases than others, the two measures need not produce exactly the same ordering or distance between models.

Table 7: Comparative heat map behavioural ranking

Rank	AI agent	Mean behavioural	Common Test accuracy	At reference	Below reference	Not evaluable	Pipeline success
1	Claude sonnet 4.6	88.97%	87.10%	34	5	11	39
2	ChatGPT 5.5	84.86%	84.52%	29	6	15	35
3	Deepseek	83.75%	82.65%	33	7	10	42
4	Groq llama-3.3-70b-versatile	79.64%	75.28%	28	9	13	46
5	Gemini 3 flash	77.94%	81.48%	26	8	16	39
6	Qwen 3.6 plus	74.67%	73.47%	29	11	10	42

Source: NL-FDEP Interactive Comparative Heat Map behavioural-ranking table.

Claude sonnet 4.6 is the strongest automated performer on both measures, with a mean semantic behavioural score of 88.97% and aggregate Common Test accuracy of 87.10%. It records 34 algorithms at the frozen semantic reference, five below reference and 11 not evaluable. Its lead over ChatGPT 5.5 is 4.11 percentage points on the mean semantic score and 2.58 percentage points on aggregate Common Test accuracy. The result therefore reflects a consistent lead across both automated measures rather than a ranking created by only one aggregation method. ChatGPT 5.5 ranks second on the mean semantic score at 84.86% and on Common Test accuracy at 84.52%, despite having only 35 pipeline successes and 15 not evaluable observations. This is an important distinction: its comparatively high behavioural score is achieved on a smaller evaluable subset than Deepseek or Qwen 3.6 plus. Strong behavioural performance where evaluation is possible does not imply that the representation is consistently processable across the complete task set. Deepseek records a mean semantic score of 83.75% and Common Test accuracy of 82.65%. It produces 42 pipeline successes and only ten not evaluable cases, giving it broader evaluability than ChatGPT 5.5 while remaining slightly lower on both automated behavioural measures. Deepseek therefore represents a different reliability profile: greater progression through the pipeline, but marginally weaker aggregate behavioural alignment among the resulting executable cases.

The middle ordering is particularly informative. Groq llama-3.3-70b-versatile records the highest pipeline success count of all six models, 46 out of 50, yet its mean semantic behavioural score is 79.64% and its aggregate test accuracy is 75.28%. Gemini 3 flash has fewer pipeline successes, 39, and more not evaluable observations, 16, but its Common Test accuracy is higher at 81.48%. The ranking therefore demonstrates directly that pipeline traversal and behavioural correctness are not interchangeable measures. A model can produce representations that are structurally processable without those representations consistently satisfying the required behavioural contract.

Qwen 3.6 plus also demonstrates this separation. It produces 42 pipeline successes and 40 evaluable algorithm results, matching Deepseek on evaluable coverage, but has the lowest mean semantic behavioural score (74.67%) and Common Test accuracy (73.47%). Its eleven below reference algorithms are the highest count in the study. In this case, the principal weakness is therefore not simply inability to traverse the pipeline; a substantial part of the difference occurs after an evaluable representation has been produced.

Comparative Radial Dashboard: Reference and Pipeline Outcomes

The comparative radial dashboard exposes the same model set through two categorical views: comparison with the frozen semantic reference and pipeline outcome. Reading these views together is useful because they show whether a model loses observations through processing failure or instead reaches executable form but produces weaker behaviour.

Table 8: Comparative radial dashboard outcomes

AI agent	Mean semantic	Test accuracy	At ref.	Below ref.	Not eval.	Success	Failure	Expected fail.	Success rate
Claude sonnet 4.6	88.97%	87.10%	34	5	11	39	10	1	78%
ChatGPT 5.5	84.86%	84.52%	29	6	15	35	15	0	70%
Deepseek	83.75%	82.65%	33	7	10	42	8	0	84%
Groq llama-3.3-70b-versatile	79.64%	75.28%	28	9	13	46	4	0	92%
Gemini 3 flash	77.94%	81.48%	26	8	16	39	11	0	78%
Qwen 3.6 plus	74.67%	73.47%	29	11	10	42	5	3	84%

Source: NL-FDEP Comparative Radial Dashboard.

The pipeline success range is 70% to 92%. Groq has the highest structural progression rate at 92%, followed by Deepseek and Qwen 3.6 plus at 84%, Claude sonnet 4.6 and Gemini 3 flash at 78%, and ChatGPT 5.5 at 70%. The behavioural ranking is different. Claude is first behaviourally despite ranking fourth jointly on pipeline success, while Groq is first on pipeline success but fourth on mean semantic behaviour. This is one of the clearest empirical findings in the study: successful transformation into executable form is necessary for the principal behavioural evaluation, but it is not evidence that the model generated algorithm is semantically correct. The difference between pipeline success and behavioural evaluability is also visible within individual models. Groq records 46 pipeline successes but only 37 evaluable semantic comparisons; Gemini records 39 successes but 34 evaluable comparisons; Qwen records 42 successes but 40 evaluable comparisons. These gaps show that reaching a nominal successful pipeline state is not sufficient to guarantee that the behavioural evidence is adequate for semantic comparison. Conversely, Claude has 39 pipeline successes and 39 evaluable comparisons, producing the strongest alignment between structural progression and behavioural evidence among the six datasets.

The expected failure category is limited to four observations in total: one for Claude sonnet 4.6 and three for Qwen 3.6 plus. These outcomes remain separate from ordinary failures because they reflect a recognised terminal condition rather than an unqualified pipeline failure. Keeping the category distinct prevents expected limitations from being silently merged into the same count as unexpected transformation failures.

Fifty Algorithm Radial Profiles

The 50 algorithm radial profiles show how the aggregate scores are constructed. Nineteen algorithms achieve a 100% semantic behavioural score for all six models: Algorithms 3, 4, 6, 8, 10-18, 22, 24, 25, 36, 39 and 42. These tasks provide little discrimination between models because every evaluated model reaches the same maximum score.

At the opposite extreme, Algorithms 7 and 44 score 0% for all six models. Algorithm 7 asks for a sentence to be split into words and stored in a list, while Algorithm 44 is the Kadane maximum subarray task. Algorithm 19, swapping two variables without a temporary variable, also scores 0% for every evaluable model, with Gemini 3 flash recorded as not evaluable. These repeated failures are analytically stronger than an isolated model error because the pattern occurs across otherwise different model datasets.

Nine algorithms are not evaluable for any of the six models: Algorithms 29, 30, 31, 32, 35, 41, 48, 49 and 50. They include higher level text classification, topic modelling, co reference, question answering, LRU-buffer, N-Queens, recursive file search, maze pathfinding and autonomous drone specifications. The result should not be read as proof that all six models are incapable of those tasks. It establishes that the responses did not produce a behaviourally evaluable result under the frozen NL-FDEP v4.37 experimental interface. This distinction is particularly important when these cases are considered later as representation level failures rather than ordinary behavioural failures.

Several algorithms strongly discriminate between models. Algorithm 1 (anagram detection) scores 100% for ChatGPT, Claude, Deepseek and Groq but 0% for Gemini and Qwen. Algorithm 2 (decimal to binary conversion) produces 100% for Deepseek and Gemini but 20% for the other four models. Algorithm 27 (in-place matrix rotation) produces 100% for ChatGPT and Claude, 0% for Deepseek, Groq and Qwen, and is not evaluable for Gemini. Algorithm 47 (inventory updates) produces the same broad split: ChatGPT and Claude score 100%, Deepseek, Groq and Qwen score 0%, and Gemini is not evaluable. These tasks demonstrate why a single model level mean is insufficient: different models reach their aggregate scores through different algorithm specific strengths and failures.

Algorithm 23 is also notable because none of the six models reaches 100%. ChatGPT, Claude, Deepseek and Gemini score 50%, while Groq and Qwen score 66.67%. Unlike the all or nothing patterns visible elsewhere, this algorithm produces a consistently partial result across all six models. It therefore represents a useful candidate for artefact level inspection because the difficulty appears common but not completely catastrophic.

Table 9: Composition of the 50-algorithm behavioural profiles

AI agent	Evaluable	100%	0%	Partial score	Not evaluable
ChatGPT 5.5	35	29	4	2	15
Claude sonnet 4.6	39	34	3	2	11
Deepseek	40	33	6	1	10
Gemini 3 flash	34	26	7	1	16
Groq llama-3.3-70b-versatile	37	28	6	3	13
Qwen 3.6 plus	40	29	9	2	10

Source: NL-FDEP All AI Agents - 50-Algorithm Radial Profiles.

Table 10 Complete 50-algorithm semantic behavioural score matrix

Alg.	Algorithm specification	ChatGPT 5.5	Claude sonnet 4.6	Deepseek	Gemini 3 flash	Groq	Qwen
1	Determine whether two given strings are anagrams of each other	100.00%	100.00%	100.00%	0.00%	100.00%	0.00%
2	Convert a non-negative decimal integer into its binary string representation	20.00%	20.00%	100.00%	100.00%	20.00%	20.00%
3	Determine whether a given year satisfies the leap year rules	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
4	Find the intersection of two sorted arrays	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
5	Remove punctuation from a sentence	NE	NE	0.00%	0.00%	100.00%	NE
6	Find the length of a given string	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
7	Split a sentence into individual words and store them in a list	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
8	Sort a list of words in alphabetical order	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
9	Find the exact position of a target value in an unsorted list	100.00%	100.00%	100.00%	0.00%	100.00%	100.00%
10	Count how many times each vowel appears in a paragraph	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
11	Remove duplicate IP addresses and return only unique IP addresses	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
12	Calculate the sum of the main diagonal of an N × N matrix	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
13	Reverse a string without using built-in reverse functions	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
14	Find the sum of all even numbers in a list	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
15	Find the second largest number in an array	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
16	Find the GCD of two numbers using the Euclidean algorithm	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
17	Compute the sum and average of a list of numbers	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
18	Count the frequency of each character in a string	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
19	Swap two variables without using a temporary variable	0.00%	0.00%	0.00%	NE	0.00%	0.00%
20	Determine all divisors of a positive integer	100.00%	100.00%	100.00%	0.00%	100.00%	100.00%
21	Determine whether a number is even or odd	100.00%	100.00%	100.00%	100.00%	NE	100.00%
22	Compute the factorial of a positive integer	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
23	Convert numerical scores into grade categories (Pass, Merit, Distinction)	50.00%	50.00%	50.00%	50.00%	66.67%	66.67%
24	Check whether a password contains at least 8 characters, one number, and one uppercase letter	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%

Alg.	Algorithm specification	ChatGPT 5.5	Claude sonnet 4.6	Deepseek	Gemini 3 flash	Groq	Qwen
25	Find the length of the longest substring without repeating characters	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
26	Perform level-order traversal on a binary tree	0.00%	100.00%	100.00%	100.00%	100.00%	100.00%
27	Rotate a square matrix 90° clockwise in-place	100.00%	100.00%	0.00%	NE	0.00%	0.00%
28	Return the maximum value in every sliding window	100.00%	100.00%	100.00%	100.00%	100.00%	0.00%
29	Build a text classification model to categorise text	NE	NE	NE	NE	NE	NE
30	Create a topic modelling system to extract topics from a corpus	NE	NE	NE	NE	NE	NE
31	Develop a co-reference resolution system to identify relationships between pronouns and nouns	NE	NE	NE	NE	NE	NE
32	Build a question answering system based on a given text	NE	NE	NE	NE	NE	NE
33	Find the square root of a non-negative integer using binary search	100.00%	NE	100.00%	100.00%	60.00%	100.00%
34	Find all common elements in two unsorted lists without duplicates	100.00%	100.00%	100.00%	100.00%	0.00%	100.00%
35	Design a memory buffer that discards the least recently accessed item when full	NE	NE	NE	NE	NE	NE
36	Find a target element in a rotated sorted array	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
37	Implement a stack using an array	NE	100.00%	NE	NE	NE	0.00%
38	Find the first non-repeating character in a string	NE	100.00%	100.00%	0.00%	100.00%	100.00%
39	Detect a cycle in a singly linked list (Floyd's Cycle Detection)	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
40	Implement a queue using two stacks	NE	100.00%	100.00%	NE	NE	0.00%
41	Solve the N Queens Problem for an arbitrary board size N	NE	NE	NE	NE	NE	NE
42	Find all pairs in an array that sum to a given target value	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
43	Solve the Tower of Hanoi problem for N discs	100.00%	100.00%	100.00%	NE	NE	100.00%
44	Find the contiguous subarray with the maximum sum (Kadane's Algorithm)	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
45	Create instructions for adding two matrices of equal size	NE	100.00%	100.00%	100.00%	100.00%	100.00%
46	Develop a process for multiplying two matrices	NE	100.00%	100.00%	NE	100.00%	100.00%
47	Update inventory quantities after purchases and deliveries	100.00%	100.00%	0.00%	NE	0.00%	0.00%
48	Search through nested folders recursively to locate a specific file type	NE	NE	NE	NE	NE	NE
49	Create a pathfinding solution through a maze using graph traversal	NE	NE	NE	NE	NE	NE
50	Develop algorithms enabling autonomous drones to coordinate movement while avoiding collisions	NE	NE	NE	NE	NE	NE

Source: NL-FDEP All AI Agents - 50-Algorithm Radial Profiles. NE = not evaluable.

Human Review Dashboard

Human Review is complete for all 300 model algorithm observations. The dashboard records 4,800 completed criterion assessments out of 4,800, with no partial or pending algorithm reviews. Across the complete review set, the mean Human Rubric is 95.09%, mean automated-human combined rubric is 91.24%, mean agreement is 92.83%, and 109 criterion level semantic issues are recorded. This gives the Human Review analysis complete coverage rather than a sample of the experimental outputs.

Table 11: Human Review dataset summary

Dataset	Complete reviews	Criteria	Mean human rubric	Mean combined rubric	Mean agreement	Semantic issues
ChatGPT 5.5	50/50	800/800	94.98%	89.84%	94.25%	25
Claude sonnet 4.6	50/50	800/800	97.08%	92.32%	92.50%	10
Deepseek	50/50	800/800	96.52%	92.19%	93.75%	32
Gemini 3 flash	50/50	800/800	93.34%	89.26%	92.25%	20
Groq llama-3.3-70b-versatile	50/50	800/800	92.77%	91.66%	90.38%	11
Qwen 3.6 plus	50/50	800/800	95.83%	92.21%	93.88%	11

Source: NL-FDEP Combined Human Review Dashboard dataset review summary.

Claude sonnet 4.6 receives the highest mean Human Rubric at 97.08% and also has the lowest dataset level semantic issue count, that of 10. Deepseek follows closely at 96.52%, while Qwen 3.6 plus records 95.83%. This human ordering differs substantially from the automated behavioural ordering, particularly for Qwen, which is last on both automated behavioural measures but third on the Human Rubric. The difference does not mean that one evidence source is correct and the other is wrong: they assess different properties. It does show that the automated behavioural measures and structured Human Review are not interchangeable.

Deepseek provides another useful example. It records the second highest Human Rubric, 96.52%, but the highest semantic-issue count, 32. The issue count therefore should not be interpreted as a simple failure count. It is a criterion level diagnostic count across 800 completed criteria for that dataset; a model can accumulate localised semantic concerns while still obtaining a high overall rubric score across the wider set of assessed properties.

Agreement between automated and human assessment is high overall but not uniform. ChatGPT 5.5 has the highest dataset level mean agreement at 94.25%, followed by Qwen at 93.88% and Deepseek at

93.75%. Groq has the lowest agreement at 90.38%. The range is relatively narrow at model level, but the criterion level analysis shows that disagreement is concentrated in particular stages rather than spread evenly through the pipeline.

Table 12: Human reviewer summary

Reviewer	Reviews	Completed	Partial	Datasets	Mean human rubric	Mean agreement	Semantic issues
Alan Johnson	300	300	0	6	95.09%	92.83%	109

Source: NL-FDEP Combined Human Review Dashboard reviewer summary.

Table 13: Criterion level Human Review analysis

Criterion	Stage	Question	Mean human score	Agreement	Semantic issues
A1	AST Semantic Alignment	Does every semantically relevant construct in the canonical pseudocode have an appropriate AST representation without missing or spurious executable nodes?	1.98/2	100.00%	2
A2	AST Semantic Alignment	Does the AST correctly represent the operands, operators, literals, indexing, collection access and call relationships of the canonical pseudocode?	1.96/2	98.67%	2
A3	AST Semantic Alignment	Are branch relationships, loop bodies, returns, loop control operations, variable scope and nested declarations represented in the correct structural context?	1.96/2	97.67%	4
A4	AST Semantic Alignment	If the AST were interpreted directly, would it permit the same observable algorithmic behaviour as the canonical pseudocode?	1.98/2	98.65%	2
C1	Pseudocode Canonicalisation	Does the canonical pseudocode conform to the AJ grammar sufficiently for deterministic interpretation by the next pipeline stage?	1.95/2	96.67%	7
C2	Pseudocode Canonicalisation	Are all semantically relevant operations from the source pseudocode preserved in the canonical representation with equivalent meaning?	1.96/2	96.67%	9
C3	Pseudocode Canonicalisation	Are conditions, branches, loop bounds, iteration behaviour, state changes and termination behaviour preserved from the source?	1.93/2	94.67%	7
C4	Pseudocode Canonicalisation	Has canonicalisation avoided introducing algorithmic behaviour that was not explicitly supported by the source?	1.97/2	98.33%	1
F1	Flow Diagram Logical Structure	Are all semantically relevant actions, decisions, entry/exit points and returns represented in the flow diagram?	1.94/2	81.33%	9
F2	Flow Diagram Logical Structure	Do the diagram connections correctly represent execution sequence, branch destinations, loop back edges and exit paths?	1.91/2	91.97%	12
F3	Flow Diagram Logical Structure	For every meaningful execution path represented by the AST, is there a corresponding path through the flow diagram without additional semantically unsupported paths?	1.96/2	93.67%	7
F4	Flow Diagram Logical Structure	Do decision conditions, TRUE/FALSE routing, loop entry conditions and loop termination behaviour preserve the logic represented by the AST?	1.94/2	92.98%	8
P1	Generated Python Correctness	Is the generated Python syntactically valid and executable for inputs valid under the source algorithm's assumptions?	1.90/2	96.31%	5
P2	Generated Python Correctness	Does the generated Python produce the expected outputs and observable state changes across the predefined behavioural tests?	1.82/2	89.00%	16
P3	Generated Python Correctness	Does the generated Python preserve source defined boundary behaviour, termination behaviour, returns and errors?	1.78/2	69.23%	12
P4	Generated Python Correctness	Does the generated Python preserve the algorithm's control flow, state changes, data structure effects and observable behaviour without introducing or removing meaningful behaviour?	1.86/2	84.28%	6

Source: NL-FDEP Combined Human Review Dashboard criterion-level analysis.

Table 14: Stage-level assessment of Human Review criteria

Stage	Mean human score	Mean agreement	Semantic issues
AST Semantic Alignment	1.97/2 (98.50%)	98.75%	10
Pseudocode Canonicalisation	1.95/2 (97.62%)	96.59%	24
Flow Diagram Logical Structure	1.94/2 (96.88%)	89.99%	36
Generated Python Correctness	1.84/2 (92.00%)	84.70%	39

Source: Derived from the criterion level values in the NL-FDEP Combined Human Review Dashboard.

The stage level pattern is clear. AST Semantic Alignment has the strongest Human Review profile, averaging 1.97 out of 2 (98.50% of the maximum criterion score) and 98.75% automated human agreement. Pseudocode Canonicalisation remains similarly strong at 97.62% of the maximum score and 96.59% agreement. The principal disagreements emerge later in the chain: Flow Diagram Logical Structure averages 89.99% agreement, while Generated Python Correctness falls to 84.70%. The number of semantic issues also rises from ten across the AST criteria to 36 across the flow criteria and 39 across the Python criteria.

Criterion P3, which asks whether generated Python preserves source defined boundary behaviour, termination behaviour, returns and errors, has the lowest agreement in the complete rubric at 69.23% and the lowest mean Human Review score at 1.78 out of 2. Criterion P2, concerning expected outputs and observable state changes across predefined behavioural tests, records the largest semantic issue count, 16. Within the flow stage, F1 has only 81.33% agreement despite a high mean human score of 1.94 out of 2. This pattern is consistent with the v4.37 issues identified during Human Review, where automated checks could under credit correct flow structures because of reachability reporting and AST to flow node count differences. It does not imply that every disagreement arose from those two defects, but it shows why retaining Human Review as an independent evidence stream materially affected interpretation of the automated results.

Human Review Terminal Outcomes and the Groq Algorithm 49 Case

The Human Review dashboard reports the numerical only mean Human Rubric for completed numerical reviews. The Final Scoring Dashboard applies the fixed denominator rule defined in the methodology. For five models these values are identical because all 50 completed reviews have numerical scores. Groq llama-3.3-70b-versatile is the exception.

Groq Algorithm 49, the maze pathfinding task, reaches the explicit terminal status HUMAN_REVIEW_COMPLETE_NOT_EVALUABLE. The review is complete, but the Human Rubric is not numerically scorable. Under the failure to demonstrate rule, this observation contributes 0% to the fixed-denominator Human Review Overall. Consequently, Groq has a numerical only Human Review mean of 92.77% across 49 scored reviews, but its final fixed denominator Human Review Overall is 90.91%. The 1.86 point reduction is therefore deliberate and traceable rather than a discrepancy between dashboards.

This case also illustrates why data absence and terminal non evaluability were separated methodologically. The result is not treated as an unresolved technical absence: the review reached a documented terminal judgement. At the same time, the 92.77% numerical only mean remains available as an audit statistic so that the effect of the zero-credit rule is visible rather than hidden.

Final Scoring Dashboard and Cross-Model Ranking

The Final Scoring Dashboard combines the two automated model level measures into an Automated Overall score: the arithmetic mean of Mean Semantic Behavioural Score and Aggregate Common Test Accuracy. Because all six models have 50 out of 50 terminal Human Review outcomes, the dashboard activates the final ranking and calculates Combined Final as the mean of Automated Overall and Human Review Overall.

Table 15: Final Scoring Dashboard ranking

Final rank	Automated rank	AI agent	Mean semantic	Test accuracy	Automated overall	Human Review Overall	Combined Final
1	1	Claude sonnet 4.6	88.97%	87.10%	88.03%	97.08%	92.56%
2	3	Deepseek	83.75%	82.65%	83.20%	96.52%	89.86%
3	2	ChatGPT 5.5	84.86%	84.52%	84.69%	94.98%	89.84%
4	4	Gemini 3 flash	77.94%	81.48%	79.71%	93.34%	86.53%
5	6	Qwen 3.6 plus	74.67%	73.47%	74.07%	95.83%	84.95%
6	5	Groq llama-3.3-70b-versatile	79.64%	75.28%	77.46%	90.91%	84.19%

Source: NL-FDEP Final Scoring Dashboard. Final ranking is active because all six datasets have complete terminal Human Review coverage.

Claude sonnet 4.6 remains first after Human Review. Its Automated Overall is 88.03%, Human Review Overall is 97.08%, and Combined Final is 92.56%. The model therefore leads on the automated evidence and retains that lead when the independent human evidence is introduced.

The most important ranking change occurs immediately below Claude. ChatGPT 5.5 is second on Automated Overall at 84.69%, while Deepseek is third at 83.20%. Human Review reverses that ordering: Deepseek receives 96.52% compared with ChatGPT's 94.98%. Their Combined Final scores become 89.86% and 89.84% respectively, moving Deepseek into second place by only 0.02 percentage points. The numerical difference is extremely small, so the final ordering should not be interpreted as evidence of a substantial performance gap. Instead, the result demonstrates how the two evidence streams can alter an ordinal ranking when the automated results are already close.

Gemini 3 flash remains fourth, with an Automated Overall of 79.71%, Human Review Overall of 93.34% and Combined Final of 86.53%. Qwen 3.6 plus moves from sixth on the automated ranking to fifth overall. Its Automated Overall is the lowest in the study at 74.07%, but its Human Review Overall is 95.83%, producing a Combined Final of 84.95%. This is the largest automated to human divergence in the six model comparison, a difference of 21.76 percentage points. The result reinforces that Qwen's low automated behavioural result does not correspond to uniformly poor human assessment of the retained representations.

Groq moves from fifth on Automated Overall to sixth on the final ranking. Its Automated Overall is 77.46%; after the Algorithm 49 zero credit rule, its Human Review Overall is 90.91% and Combined Final is 84.19%. If only the 49 numerical Human Review scores were averaged, the human mean would be 92.77%, but that descriptive value is not used for the final ranking because the study retains a fixed denominator of 50 terminal outcomes.

The scoring layers also differ in their spread. Automated Overall ranges from 74.07% to 88.03%, a 13.96 point span. Human Review Overall ranges from 90.91% to 97.08%, a 6.17 point span. Combined Final therefore narrows the model level range to 8.37 points, from 84.19% to 92.56%. Human Review does not simply reproduce the automated ordering; it moderates some of the automated differences and changes the relative position of models whose automated scores are close.

The Human Review dashboard also reports a dataset level Mean Combined Rubric, but this should not be confused with Combined Final in Table 15. Mean Combined Rubric is calculated within the Human Review

rubric at the algorithm/criterion level. Combined Final is the separate study level measure used by the Final Scoring Dashboard, combining Automated Overall with the fixed denominator Human Review Overall. Keeping these two quantities conceptually separate avoids treating different aggregation levels as though they were the same score.

Results in Relation to the Research Questions

RQ1 - Process ability of generated pseudocode:

Pipeline success varies substantially between models, from 35 of 50 for ChatGPT 5.5 to 46 of 50 for Groq llama-3.3-70b-versatile. However, the highest pipeline success rate does not correspond to the highest behavioural score. The results therefore show that generating pseudocode capable of traversing the formal pipeline is one component of reliability, but not a sufficient condition for behavioural correctness.

RQ2 - Preservation across representations:

Human Review provides the strongest direct stage level evidence. AST and canonicalisation criteria show very high human scores and automated human agreement, while disagreement becomes more pronounced in the flow and generated Python stages. The decreasing agreement from 98.75% at AST level to 84.70% at generated-Python level identifies the later representations as the points at which automated and human assessments diverge most strongly. This does not by itself prove where every semantic defect originates, but it narrows the areas requiring artefact level investigation.

RQ3 - Comparative behavioural correctness:

Claude sonnet 4.6 is the strongest automated behavioural performer, with 88.97% mean semantic behaviour and 87.10% Common Test accuracy. ChatGPT 5.5 and Deepseek form the next group, while Groq, Gemini and Qwen show lower automated behavioural results. The 50 algorithm profiles demonstrate that these averages conceal substantial task level variation, including nineteen universally successful algorithms; two universally zero scoring algorithms and nine algorithms that are not evaluable for any model.

RQ4 - Additional evidence from automated rubric and Human Review:

Human Review materially changes the interpretation of the model comparison. It raises Deepseek above ChatGPT in the final combined ranking, raises Qwen above Groq, identifies a terminal non evaluable case requiring fixed denominator zero credit, and exposes concentrated disagreement in flow and generated Python criteria. The value of the additional evidence is therefore not simply another score; it identifies where automated conclusions should be inspected rather than accepted without qualification.

RQ5 - Reoccurring failure and variation patterns:

The algorithm level results identify three broad recurring patterns: tasks that all six models solve successfully, tasks that all evaluable models fail, and high discrimination tasks where performance varies sharply between models. Algorithms 1, 2, 27 and 47 are particularly clear examples of the third pattern. The nine universally not evaluable tasks form a fourth category in which the controlled interface itself becomes part of the interpretation. These groups provide a data driven basis for selecting the artefact level case studies used in the subsequent analysis.

Summary of Empirical Findings

Taken together, the dashboards show that model reliability cannot be represented adequately by one measure. Claude sonnet 4.6 provides the strongest overall result and remains first across the final combined ranking, but other models demonstrate different strengths: Groq has the highest pipeline success count, Deepseek combines broad evaluability with strong Human Review and moves to second in the final ranking, ChatGPT produces strong behavioural results on a smaller evaluable subset, and Qwen receives substantially stronger Human Review than its automated behavioural ranking would suggest.

The algorithm level evidence is equally important. Nineteen tasks do not discriminate between models because all six score 100%, while Algorithms 7 and 44 fail for all six. Several other tasks create sharp cross model splits, and nine remain non evaluable across the complete model set. Human Review then shows that semantic alignment is strongest in the early AST and canonicalisation stages, while automated human disagreement increases in the flow and generated Python stages. The empirical result is therefore not simply a ranking of six models; it is a set of distinct model level, algorithm level and representation level patterns that can be traced through the retained NL-FDEP artefacts.

Discussion

From Strict Pseudocode to a Controlled Intermediate Language

The experience of using a strict pseudocode dialect raises a wider question concerning the role of intermediate representations. Pseudocode is normally informal. It is intended primarily to help express algorithms without committing to the syntax of one programming language, and bridge the gap between natural Language and controlled coded instruction. That functionality is useful for direct interpretation of communication between machines and humans, and should as I believe it be considered as a basic formal language. That prospect goes beyond the remit of this study but if nothing else, this study has shown the relevance and importance of this under rated concept. The strict dialect used by the NL-FDEP occupies an intermediate position. It remains recognisable and readable as pseudocode, but its acceptable form is deliberately restricted so that it can be processed reliably. In the undertaking of adapting the strict dialect I chose to model it in the language of python due to the fact I was generating python; but this did not need to be the case, in fact I could have used any computer language as a template. This leads to an interesting area of exploration in my opinion; as to the future direction a later study on this subject could take, and the broader concept that can be described as a **Controlled Intermediate Language (CIL)**.

A Controlled Intermediate Language can be defined as:

A deliberately restricted intermediate representation intended to provide a consistent, human readable and machine processable interface between natural language algorithm specifications and formal structured representations.

The current NL-FDEP dialect should not be treated as proof that one universal CIL has already been identified. It provides a practical example of the underlying principle. A future CIL could seek to reduce representational variation between language models, provide predictable input for deterministic parsing, retain sufficient human readability for inspection and avoid silently inventing algorithmic meaning.

Whether such a representation improves reliability remains an empirical question.

Flow Diagram as an Alternative Controlled Representation

The architecture also broadens this issue beyond textual pseudocode. The NL-FDEP supports a structured flow diagram as an alternative input route:

Structured Flow Diagram → Machine Readable Graph → Python → Behavioural Testing

This demonstrates that an intermediate representation does not necessarily need to be textual. A flow diagram can remain understandable to a human while representing decisions; loops, sequence and branching relationships in a sufficiently constrained form for deterministic machine processing. The broader issue may therefore not be whether pseudocode itself should be standardised. It may instead be whether the **interface between human algorithmic intent and machine readable structure** can be standardised. Conceptually, this can be expressed as:

Human Specification → Controlled Intermediate Representation → Formal Machine Representation → Executable Code

Under this interpretation, restricted pseudocode and structured flow diagrams are alternative candidates for the controlled intermediate representation. The defining property is not whether the representation is textual or graphical, but whether it is sufficiently constrained to support deterministic processing while preserving understandable algorithmic intent. This opens a clear future research direction. Restricted pseudocode, graphical flow representations and potentially other graph based intermediate forms could be compared directly using the same downstream NL-FDEP stages.

Relevant measures could include parsing or import success, normalisation burden, behavioural correctness, semantic divergence, cross model variance and human comprehensibility.

Conclusion

Implications for Language Model Evaluation

The results described here suggest that evaluation of generated algorithms should not depend entirely on final code execution or textual similarity. This does not automatically establish causation, but a comparative experiment could in fact evaluate two conditions: generation using unconstrained pseudocode and generation using a formally defined controlled pseudocode language. If the controlled condition produced lower representational variance and higher downstream processing reliability without reducing the ability of models to express valid alternative algorithms, this would provide evidence that a standardised intermediate pseudocode representation could improve natural language to program transformation.

I must emphasise the findings of this study do not establish that such a universal language would necessarily improve efficiency or correctness. Instead, the use of a strict dialect within NL-FDEP identifies this as a testable hypothesis and a potentially valuable direction for subsequent research.

Implications for Compilers

Although the current NL-FDEP implementation generates executable Python as its final software artefact, the architecture suggests a broader direction for future work. If a Controlled Intermediate Representation can preserve algorithmic meaning independently of a specific programming language, the same representation could potentially support generation into multiple target languages rather than Python alone. This would move the role of the intermediate representation closer to that used in conventional compiler architectures, where a language independent internal form can be transformed into different target representations. Within the NL-FDEP, this could mean retaining the same natural language to structured representation pathway while replacing or extending the final code generation stage with language specific back ends.

For example:

Natural Language

↓

Controlled Intermediate Representation

↓

Machine-Readable Flow / Graph Representation

↓

Language-Specific Code Generator

↓

Python | Java | C# | C++ | JavaScript | Other Target Languages

Such an extension would allow future studies to investigate whether semantic preservation remains stable when the same controlled representation is compiled into different programming languages. It would also enable comparison of whether particular target languages introduce additional constraints, ambiguity or transformation complexity.

This raises a broader research question:

Can a sufficiently controlled intermediate representation provide a language independent semantic interface from which equivalent executable implementations can be generated reliably across multiple programming languages?

The current study does not test this capability directly, because Python is the only executable target used within the experimental configuration. However, the separation between pseudocode, AST, flow representation and code generation provides an architectural basis from which multi language output could be investigated in future work.

Acknowledgements

The comparative evaluation used six transformer-based language models: GPT-5.5 (OpenAI, 2026), Claude Sonnet 4.6 (Anthropic, 2026), DeepSeek (DeepSeek, 2026), Gemini 3 Flash (Google, 2025), Llama 3.3 70B Versatile served through Groq (Groq, 2026), and Qwen3.6-Plus (Alibaba Cloud, 2026). Each model was evaluated against the same set of 50 natural language algorithm specifications under the frozen experimental conditions described previously.

OpenAI ChatGPT was used during the development of the NL-FDEP pipeline as a supporting development tool. Assistance included discussion of software architecture, debugging, code refinement, test design, validation logic, data processing approaches, dashboard development and documentation. Suggestions generated through ChatGPT were reviewed, tested and adapted by the author before incorporation into the software. Responsibility for the pipeline architecture, implementation decisions, experimental design, validation procedures and final software remained with the author.

ChatGPT was also used during preparation of the written report to assist with language refinement, restructuring, and presentation of material written and supplied by the author. It was also used to assist with interpretation and presentation of the completed experimental results. All research decisions, experimental design, software development, data collection, Human Review, verification of numerical

results and final academic judgements remained the responsibility of the author. The ChatGPT assistance used during final preparation included GPT-5.6 Sol (OpenAI, 2026).

The ChatGPT assistance used during the final development and writing stages included GPT-5.6 Sol (OpenAI, 2026). This use was separate from GPT-5.5, which formed one of the six transformer based language models evaluated as part of the experimental study.

To ensure academic integrity during the development of the NL-FDEP great care was taken to avoid unnecessary bias of the processed inputs an extensive audit was undertaken on all 300 examples used in this study to ensure that semantic meaning was preserved and functionality was Not invented the principal design choices I made included that each algorithm was read and interpreted by myself and any decision as to how the code could be interpreted was made solely on the assumption that no semantic misalignment took place it was therefore necessary to adopt the case by case approach. To maintain traceability and a clear fall-back position in the event of a catastrophic failure I ensured a new vertion was generated at each new instance of this process. The software, supporting project artefacts and materials associated with NL-FDEP are maintained in a public GitHub repository (Johnson, 2026). The repository provides a persistent project location through which the implementation and associated research materials can be inspected alongside the methodological description presented here; as such I have chosen not to include an appendix section with this report all dashboards and artefacts can be accessed via the repository.

One of the trusted baseline text processing example used an adapted excerpt from Chapter I, ‘The River Bank’, of Kenneth Grahame’s *The Wind in the Willows* (Grahame, 1908). The passage was used as fixed textual input for the relevant baseline algorithm and was not generated by a language model.

Microsoft Copilot was used as a supplementary verification tool during development of the trusted baseline examples (Microsoft, 2026). It was used to independently review selected baseline logic and expected outcomes as an additional checking mechanism and in the production of some of the later examples. Copilot outputs were not treated as authoritative evidence; baseline examples were subsequently validated through the NL-FDEP testing and study-readiness procedures.

References

Aho, A.V., Lam, M.S., Sethi, R. and Ullman, J.D. (2006) Compilers: Principles, Techniques, and Tools. 2nd edn. Boston, MA: Addison-Wesley.

Ammann, P. and Offutt, J. (2016) Introduction to Software Testing. 2nd edn. Cambridge: Cambridge University Press.

Austin, J. et al. (2021) 'Program Synthesis with Large Language Models', arXiv:2108.07732.

Chen, L. et al. (2024) 'A Survey on Evaluating Large Language Models in Code Generation Tasks', arXiv:2408.16498.

Chen, M. et al. (2021) 'Evaluating Large Language Models Trained on Code', arXiv:2107.03374.

Gulwani, S., Polozov, O. and Singh, R. (2017) 'Program Synthesis', Foundations and Trends in Programming Languages, 4(1-2), pp. 1-119.

Jiang, J. et al. (2024) 'A Survey on Large Language Models for Code Generation', arXiv:2406.00515.

Jimenez, C.E. et al. (2024) 'SWE-bench: Can Language Models Resolve Real-World GitHub Issues?', International Conference on Learning Representations (ICLR 2024).

Samoaa, H.P., Bayram, F., Salza, P. and Leitner, P. (2022) 'A systematic mapping study of source code representation for deep learning in software engineering', IET Software, 16(4), pp. 351-385.

Vaswani, A. et al. (2017) 'Attention Is All You Need', Advances in Neural Information Processing Systems, 30.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B. and Wesslén, A. (2012) Experimentation in Software Engineering. Berlin, Heidelberg: Springer.

Cambridge University Press & Assessment (2023) *Pseudocode Guide for Teachers: Cambridge International AS & A Level Computer Science 9618 (For exams in 2026)*. Cambridge: Cambridge University Press & Assessment. Available at: https://www.cambridgeinternational.org/Images/697401-2026-pseudocode-guide-for-teachers.pdf

OpenAI (2026) *GPT-5.5*. OpenAI. Available at: OpenAI Developers, GPT-5.5 model documentation. OpenAI Developers

Anthropic (2026) *Claude Sonnet 4.6*. Anthropic, 17 February. Available at: Anthropic. Anthropic

DeepSeek (2026) *DeepSeek model documentation*. DeepSeek. Available at: DeepSeek Transparency Center. DeepSeek

Google (2025) *Gemini 3 Flash*. Google, 17 December. Available at: Google Gemini. blog.google

Groq (2026) *Llama 3.3 70B Versatile (llama-3.3-70b-versatile)*. GroqDocs. Available at: Groq. GroqCloud

Alibaba Cloud (2026) *Qwen3.6-Plus*. Alibaba Cloud Model Studio. Available at: Alibaba Cloud.

OpenAI (2026) *ChatGPT, GPT-5.6 Sol* [Large language model]. OpenAI. Available at: ChatGPT.

Johnson, A. (2026) *Final_Project: MSc Computer Science Final Project* [GitHub repository]. GitHub. Available at: github.com/ajohnson-2210/Final_Project .

Grahame, K. (1908) *The Wind in the Willows*. Chapter I, 'The River Bank'. Project Gutenberg. Available at: https://www.gutenberg.org/ebooks/27805.

Microsoft (2026) *Microsoft Copilot* [Generative AI tool]. Microsoft. Available at: Microsoft Copilot.